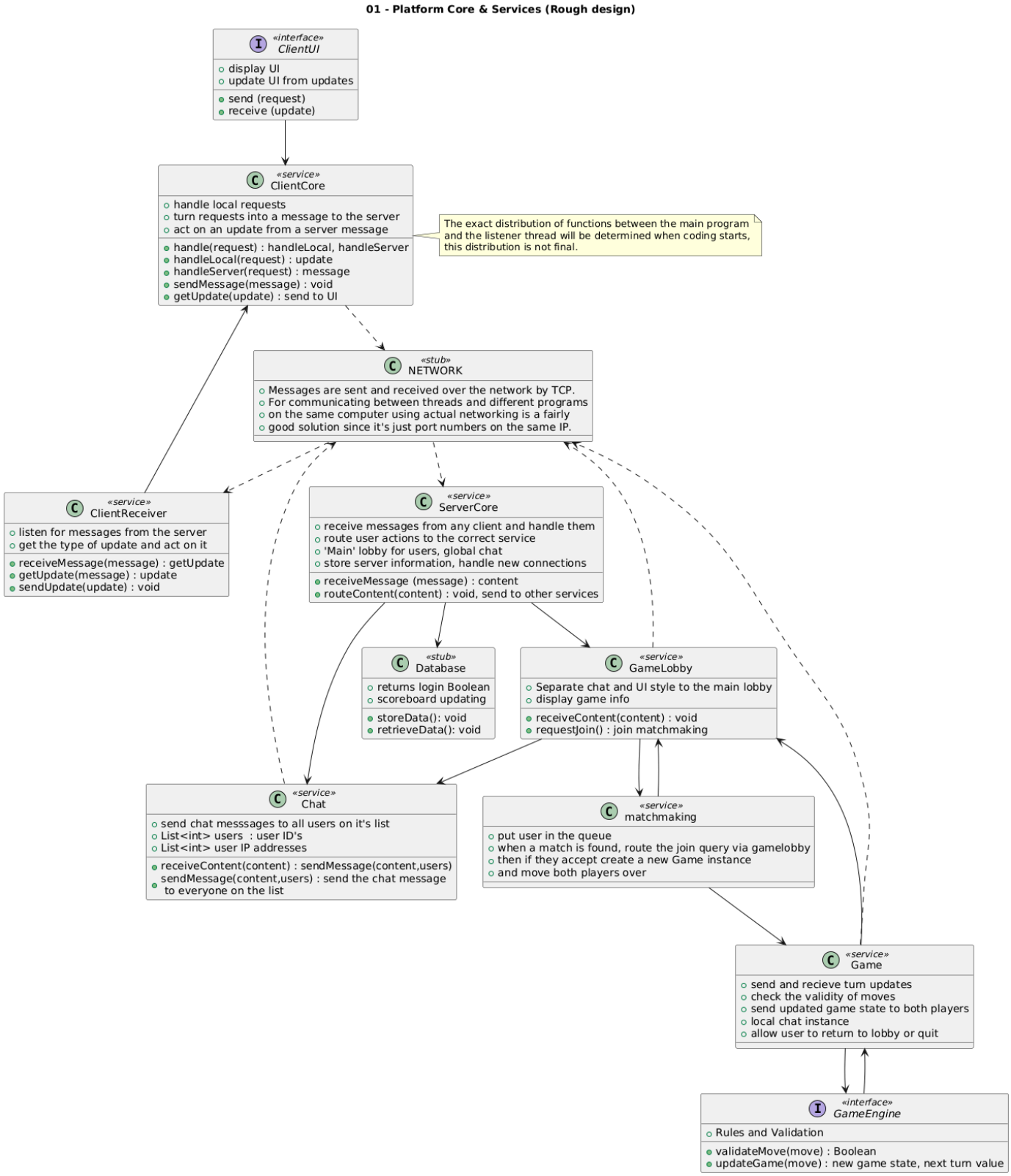
UML Class Diagram Overview

SENG300 | Online Multiplayer Game Platform — Project Iteration 1

This document provides a structured overview of the four class diagrams designed for the platform. Together they cover the full system architecture across four domains: platform networking and services, player data and persistence, game logic abstractions, and the client-side UI layer.

Diagram 01 — Platform Core & Services

File: `02.01_Platform_Core_and_Services.png`



Purpose

This diagram defines the networking backbone and server-side service architecture of the platform. It specifies how clients communicate with the server, how server-side services are routed, and where game session management, matchmaking, chat, and data persistence connect.

Key Classes & Interfaces

`ClientUI` **(Interface)** The top-level interface for any user-facing view. Defines the contract between the UI layer and the client-side core: displaying the UI, receiving state updates from the server, and sending requests upward.

`display UI`	Renders current state to the player
`update UI from updates`	Reacts to incoming server-side updates
`send(request)`	Forwards player actions to ClientCore
`receive(update)`	Accepts updates delivered from ClientCore

`ClientCore` **(Service)** Central client-side coordinator. Handles all local requests, serializes them into server messages, and processes incoming server updates to dispatch back to the UI.

Method	Purpose
`handle(request)`	Routes to either handleLocal or handleServer
`handleLocal(request)`	Processes requests that can be resolved client-side
`handleServer(request)`	Serializes and sends requests to the server
`sendMessage(message)`	Dispatches outgoing network message
`getUpdate(update)`	Forwards incoming server update to the UI

`NETWORK` **(Stub)** Represents the TCP networking layer. Messages are sent and received over the network. Noted as viable for same-machine communication via port numbers on the same IP — enabling local multiplayer testing without a remote server. This is a stub; the actual implementation will be handled by the database/networking team.

Method	Purpose
`storeData()`	Persists data to network layer
`retrieveData()`	Fetches data from network layer

`ClientReceiver` **(Service)** Listens for incoming messages from the server on a background thread. Identifies the type of update and dispatches it appropriately.

Method	Purpose
`receiveMessage(message)`	Accepts raw incoming server message
`getUpdate(message)`	Extracts typed update from message

Method	Purpose
`sendUpdate(update)`	Forwards parsed update to ClientCore

`ServerCore` **(Service)** The central server-side router. Receives messages from any client, routes actions to the appropriate service, maintains the global lobby, handles chat, and manages new connections.

Method	Purpose
`receiveMessage(message)`	Accepts incoming client message
`routeContent(content)`	Dispatches content to the correct service

`Database` **(Stub)** Server-side persistence stub. Returns login booleans and handles scoreboard updates. The real implementation will be swapped in without structural changes.

Method	Purpose
`storeData()`	Writes data to persistent storage
`retrieveData()`	Reads data from persistent storage

`GameLobby` **(Service)** Maintains a separate chat and UI context from the main lobby. Displays available game info and routes players into matchmaking.

Method	Purpose
`receiveContent(content)`	Receives lobby-level events
`requestJoin()`	Triggers the matchmaking queue for the player

`Chat` **(Service)** Manages chat broadcasting across all users on its subscriber list. Maintains user IDs and IP addresses for delivery.

Method	Purpose
`receiveContent(content)`	Accepts a chat message
`sendMessage(content, users)`	Broadcasts message to all listed users

`matchmaking` **(Service)** Queues players and pairs them when a match is found. Routes the join query via GameLobby, creates a new Game instance, and moves both players into it.

`Game` **(Service)** Manages the active game session server-side. Sends and receives turn updates, validates moves, broadcasts updated game state to both players, provides a local chat instance, and handles player return-to-lobby or quit.

`GameEngine` **(Interface)** The rules-and-validation contract for all game logic. Stateless — receives a move and returns a boolean (validity) or the new game state and next turn value.

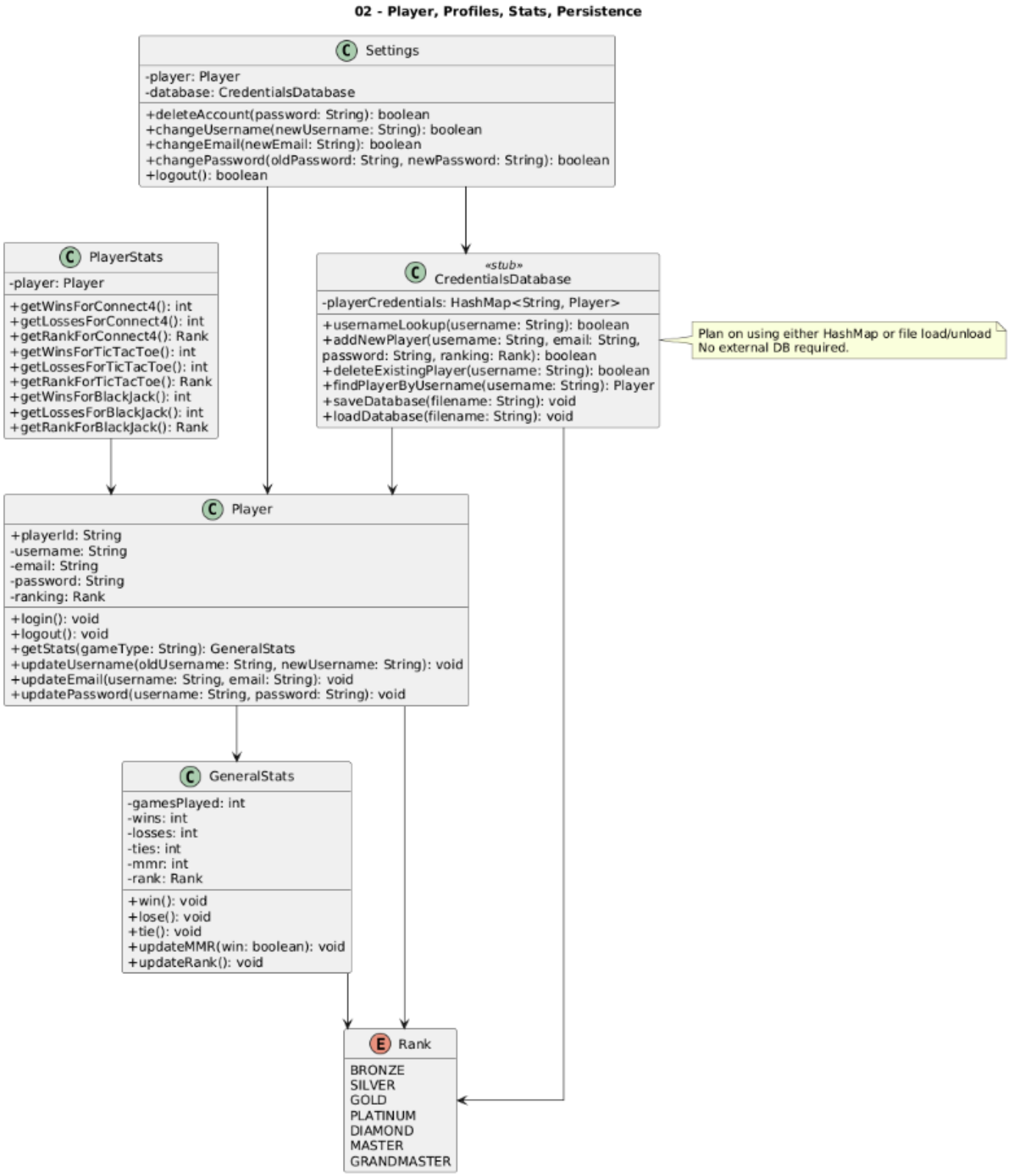
Method	Purpose
`validateMove(move)`	Returns boolean — is the move legal?
`updateGame(move)`	Returns new game state and next turn value

Key Relationships

- `ClientUI` → `ClientCore` → `NETWORK` → `ServerCore` (the primary request chain)
- `ClientReceiver` ← `NETWORK` (receives server-pushed updates asynchronously)
- `ServerCore` → `Database` , `GameLobby` , `Chat` , `matchmaking` (routes to each service)
- `matchmaking` → `GameLobby` → `Game` → `GameEngine` (game session creation chain)
- `Game` ↔ `GameEngine` (bidirectional: game delegates validation and state updates)

Diagram 02 — Player, Profiles, Stats & Persistence

File: `02.02 - Player, Profiles, Stats, Persistence, Modified.png`
Note from diagram: Plans to use either HashMap or file load/unload. No external DB required.



Purpose

This diagram defines all persistent player data, account management, and statistics tracking. It covers the Player domain model, per-game statistics, ranking, and the account settings/credential management system.

Key Classes

`Player` The central domain object representing an authenticated user of the platform.

Field/Method	Description
`playerId: String`	Unique platform identifier
`username: String`	Display name
`email: String`	Account email
`password: String`	Stored credential
`ranking: Rank`	Current rank enum value
`login() / logout()`	Session lifecycle management
`getStats(gameType: String)`	Returns GeneralStats for a specific game
`updateUsername() / updateEmail() / updatePassword()`	Account mutation methods

`GeneralStats` Tracks win/loss/tie/MMR data for a player within a specific game type.

Field/Method	Description
`gamesPlayed: int`	Total games completed
`wins: int , losses: int , ties: int`	Outcome counters
`mmr: int`	Matchmaking rating score
`rank: Rank`	Current rank within this game context
`win() , lose() , tie()`	Outcome recording methods
`updateMMR(win: boolean)`	Adjusts MMR based on game result
`updateRank()`	Recalculates rank from MMR

`PlayerStats` A convenience aggregation class that exposes per-game win/loss/rank data through named accessors, organized by game type (Connect4, TicTacToe, Blackjack).

Representative Methods	
`getWinsForConnect4()`	Wins in Connect Four
`getRankForConnect4()`	Rank in Connect Four
`getWinsForTicTacToe()`	Wins in Tic-Tac-Toe
`getWinsForBlackJack()`	Wins in Blackjack

Representative Methods	
(and corresponding losses/rank methods)	

`CredentialsDatabase` **(Stub)** Simulates the user authentication database using a `HashMap<String, Player>`. Provides the stubs for all account operations that will later connect to a real persistence layer.

Method	Description
`usernameLookup(username)`	Returns boolean — does username exist?
`addNewPlayer(username, email, password, ranking)`	Registers a new player
`deleteExistingPlayer(username)`	Removes a player account
`findPlayerByUsername(username)`	Returns the Player object
`saveDatabase(filename)`	Serializes to file
`loadDatabase(filename)`	Deserializes from file

`Settings` Account management UI-facing class. Holds references to the active Player and CredentialsDatabase to execute account changes.

Method	Description
`deleteAccount(password)`	Deletes account after password confirmation
`changeUsername(newUsername)`	Updates display name
`changeEmail(newEmail)`	Updates account email
`changePassword(oldPassword, newPassword)`	Updates credential with verification
`logout()`	Ends the player session

`Rank` **(Enum)** Defines the platform's ranking ladder used by both `GeneralStats` and `Player`.

```
BRONZE → SILVER → GOLD → PLATINUM → DIAMOND → MASTER → GRANDMASTER
```

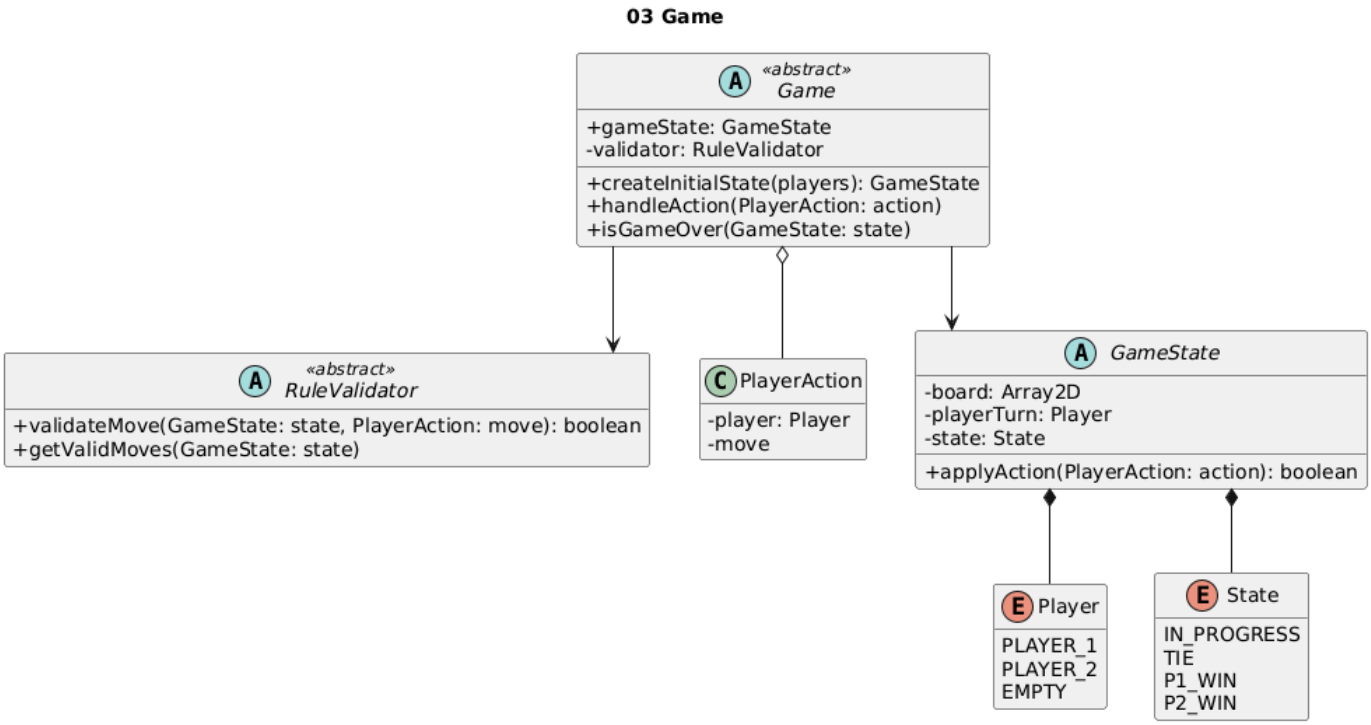
Key Relationships

- `Settings` depends on `Player` and `CredentialsDatabase` — account changes flow through Settings
- `PlayerStats` depends on `Player` — aggregates that player's stats per game
- `Player` owns `GeneralStats` (via `getStats()`) — one Player can have stats for multiple game types

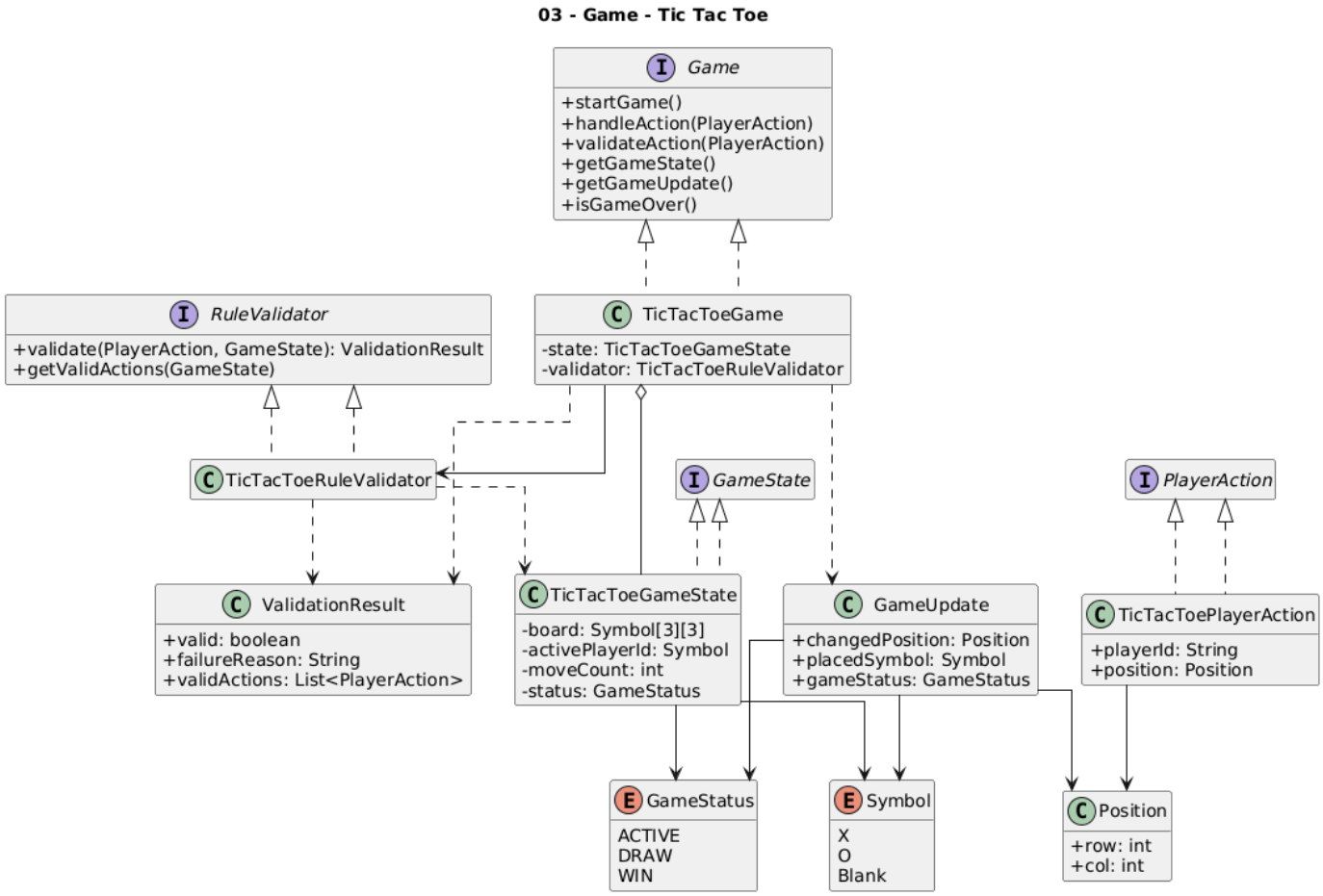
- `GeneralStats` uses `Rank` — rank is computed from MMR
- `Player` uses `Rank` — overall platform rank stored on the player object

Diagram 03 — Game Layer

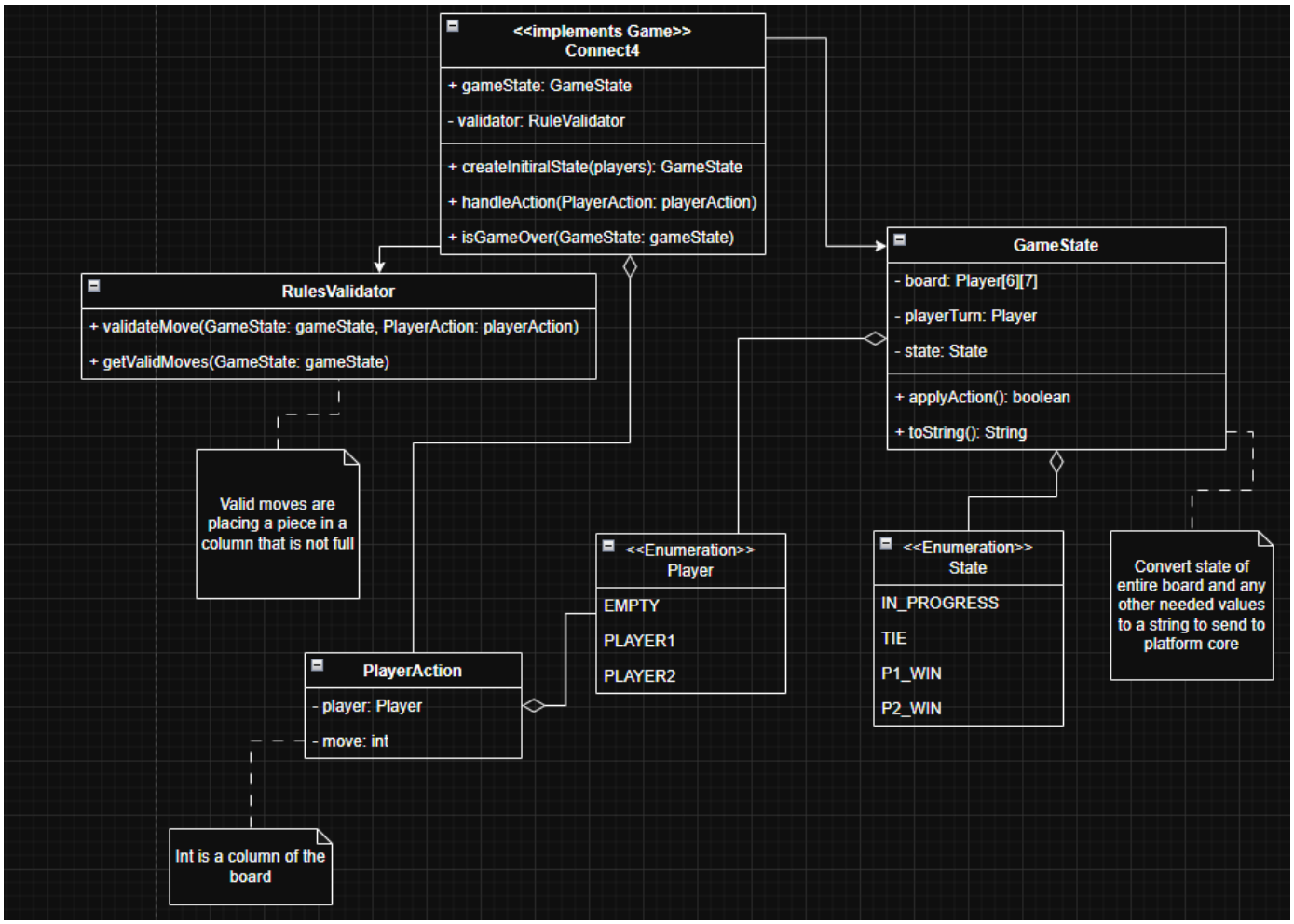
File: `Game Class Diagram.png`



File: 02.03 – Tic Tac Toe Class Diagram.png



File: `Connect4ClassDiagram.png`



Purpose

Defines the abstract game framework that all sample games (Tic-Tac-Toe, Connect Four, etc.) must implement. Enforces a clean separation between platform-level session management and game-specific logic, ensuring games are interchangeable within the platform's turn engine.

Key Classes & Interfaces

`Game` **(Abstract Class)** The top-level abstraction for any game on the platform. Concrete game implementations extend this class and supply their own game-specific logic.

Field/Method	Description
`gameState: GameState`	Current authoritative game state
`validator: RuleValidator`	Injected rule-checking component
`createInitialState(players)`	Initializes a fresh game state for the given players
`handleAction(action: PlayerAction)`	Processes a player's submitted action

Field/Method	Description
`isGameOver(state: GameState)`	Checks terminal conditions for the current state

`RuleValidator` **(Abstract Class)** Encapsulates all game-specific rule logic, keeping it independent from the Game class and fully testable in isolation.

Method	Description
`validateMove(state, move)`	Returns boolean — is this action legal given the current state?
`getValidMoves(state)`	Returns all legal moves available from the current state

`GameState` **(Abstract Class)** Represents the complete, authoritative state of a game at any point in time. Concrete implementations store the board representation appropriate for their game type.

Field/Method	Description
`board: Array2D`	2D array representing the current board
`playerTurn: Player`	Which player (PLAYER_1 or PLAYER_2) acts next
`state: State`	Current game status (IN_PROGRESS, TIE, P1_WIN, P2_WIN)
`applyAction(action: PlayerAction)`	Applies a validated action to produce the new state

`PlayerAction` A lightweight data object representing a single player's move submission.

Field	Description
`player: Player`	The player submitting the action (PLAYER_1 or PLAYER_2)
`move`	The move data (position, piece, etc. — game-specific)

`Player` **(Enum)** Identifies which player slot is acting. Distinct from the `Player` domain class in Diagram 02.

Value	Meaning
`PLAYER_1`	First player in the session
`PLAYER_2`	Second player in the session
`EMPTY`	No player (used for unoccupied board positions)

`State` **(Enum)** Represents the outcome or progression status of a game session.

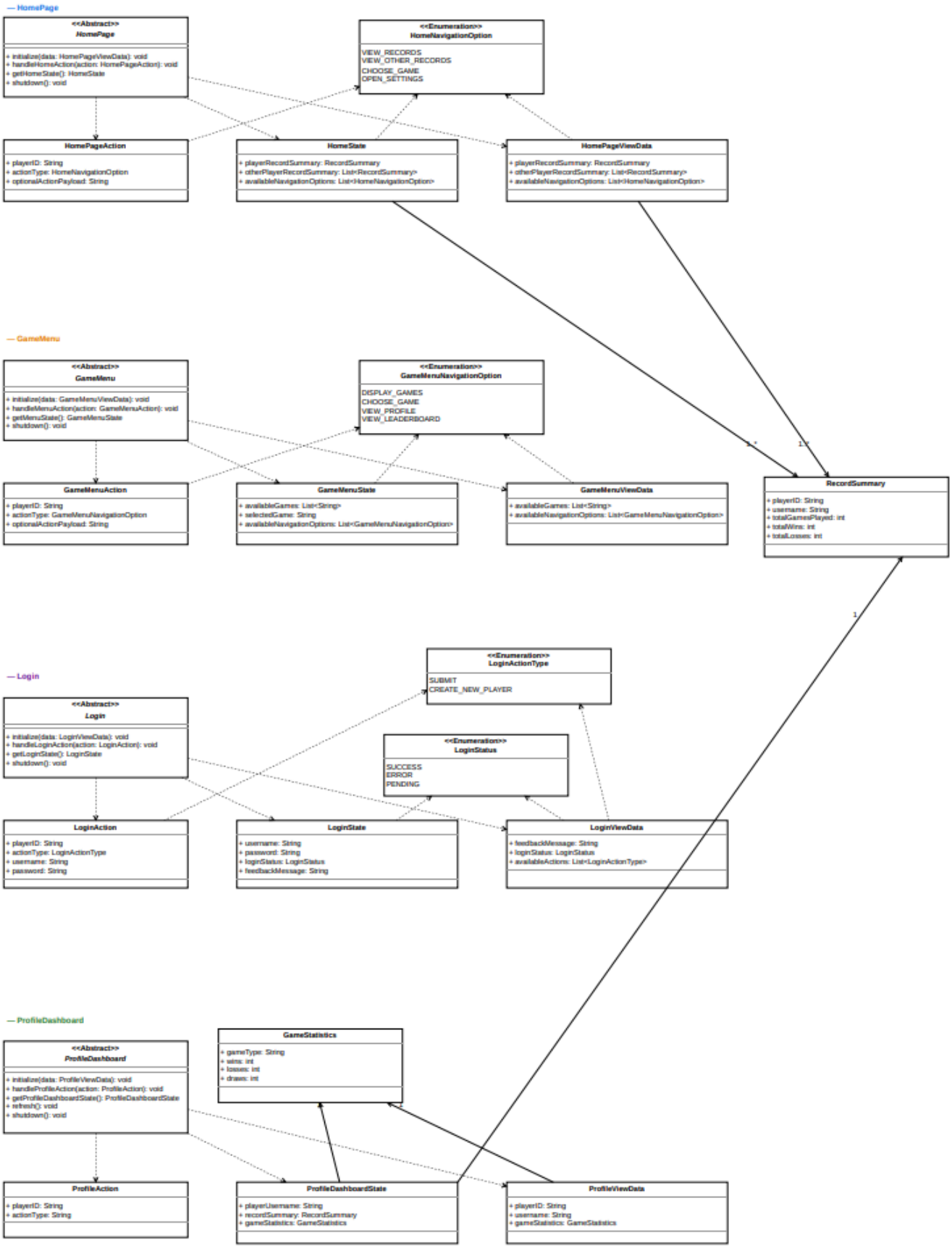
Value	Meaning
`IN_PROGRESS`	Game is ongoing
`TIE`	Game ended in a draw
`P1_WIN`	Player 1 has won
`P2_WIN`	Player 2 has won

Key Relationships

- `Game` aggregates `GameState` — the game owns and manages the current state
- `Game` aggregates `RuleValidator` — injected at construction, enabling different rule sets per game type
- `Game` uses `PlayerAction` — receives actions and passes them to RuleValidator for validation
- `GameState` uses `PlayerAction` via `applyAction()` — state transitions are driven by validated actions
- `GameState` uses the `Player` and `State` enums — tracks turn ownership and outcome

Diagram 04 — UI Class Diagram

File: `UI_Class_Diagram.png`



Purpose

Defines the client-side view layer architecture for all major screens of the platform. Each screen follows a consistent pattern: an abstract view class, a view data input object, an action object, and a state output object. This pattern enforces clean separation between the UI rendering concern and the data/logic layers.

Shared Data Types

`RecordSummary` — Lightweight player record used across multiple screens.

Field	Description
`playerID: String`	Player identifier
`username: String`	Display name
`totalGamesPlayed: int`	Total games across all types
`totalWins: int`	Total wins across all types
`totalLosses: int`	Total losses across all types

`GameStatistics` — Per-game-type stats for display in the profile.

Field	Description
`gameType: String`	Name of the game (e.g., "TicTacToe")
`wins: int`	Wins for this game type
`losses: int`	Losses for this game type
`draws: int`	Draws for this game type

Screen: `HomePage` (Abstract)

The main landing screen after login. Displays the player's record summary, allows navigation to game selection, settings, and profile viewing.

Component	Description
`initialize(HomePageViewData)`	Populates screen from backend data
`handleHomeAction(HomePageAction)`	Processes player navigation choice
`getHomeState()`	Returns current rendered state
`shutdown()`	Cleans up the view

`HomePageAction` carries a `playerID`, an `actionType` (`HomeNavigationOption`), and an optional payload string.

`HomeState` holds the active player's `RecordSummary`, a list of other players' `RecordSummary` objects, and available navigation options.

`HomeNavigationOption` **(Enum):** `VIEW_RECORDS` · `VIEW_OTHER_RECORDS` · `CHOOSE_GAME` · `OPEN_SETTINGS`

Screen: `GameMenu` (Abstract)

The game selection screen. Displays available games and allows the player to choose one, view their profile, or access the leaderboard.

Component	Description
`initialize(GameMenuViewData)`	Sets up the menu with available games
`handleMenuAction(GameMenuAction)`	Processes game or navigation selection
`getMenuState()`	Returns current rendered menu state
`shutdown()`	Cleans up the view

`GameMenuAction` carries a `playerID`, `actionType` (`GameMenuNavigationOption`), and optional payload.

`GameMenuState` holds the list of available game names, the currently selected game, and available navigation options.

`GameMenuNavigationOption` **(Enum):** `DISPLAY_GAMES` · `CHOOSE_GAME` · `VIEW_PROFILE` · `VIEW_LEADERBOARD`

Screen: `Login` (Abstract)

The authentication screen for both login and new account creation.

Component	Description
`initialize(LoginViewData)`	Sets up the login form with initial state
`handleLoginAction(LoginAction)`	Processes credential submission or registration
`getLoginState()`	Returns current form state including feedback
`shutdown()`	Cleans up the view

`LoginAction` carries a `playerID`, `actionType` (`LoginActionType`), `username`, and `password`.

`LoginState` holds current `username`, `password` field content, `loginStatus`, and a `feedbackMessage` for error or success display.

`LoginStatus` **(Enum):** `SUCCESS` · `ERROR` · `PENDING`

`LoginActionType` **(Enum):** `SUBMIT` · `CREATE_NEW_PLAYER`

Screen: `ProfileDashboard` (Abstract)

Displays a player's complete profile including their record summary and per-game statistics. Supports refresh (e.g., after a game completes).

Component	Description
`initialize(ProfileViewData)`	Populates the dashboard with player data
`handleProfileAction(ProfileAction)`	Handles any profile-level user action
`getProfileDashboardState()`	Returns current rendered state
`refresh()`	Re-fetches and re-renders updated data
`shutdown()`	Cleans up the view

`ProfileAction` carries a `playerID` and an `actionType` string (open-ended for extensibility).

`ProfileDashboardState` holds the player's `username`, `RecordSummary`, and `GameStatistics`.

`ProfileViewData` carries `playerID`, `username`, and `GameStatistics` needed to render the screen.